Scrapy Playwright Guide: Render & Scrape JS Heavy Websites

Released by Microsoft in 2020, Playwright.js is quickly becoming the most popular headless browser library for browser automation and web scraping thanks to its cross-browser support (can drive Chromium, WebKit, and Firefox browsers, whilst Puppeteer only drives Chromium) and developer experience improvements over Puppeteer.

So it is great to see that a number of the core Scrapy maintainers developed a Playwright integration for Scrapy: scrapy-playwright

Scrapy Playwright is one of the best headless browser options you can use with Scrapy so in this guide we will go through how:

- How To Install Scrapy Playwright
- How To Use Scrapy Playwright In Your Spiders
- How To Wait For The Page To Load
- How To Scrape Multiple Pages
- How To Scroll The Page Elements With Scrapy Playwright
- How To Take screenshots With Scrapy Playwright

> **NOTE**
>
> As of writing this guide, Scrapy Playwright doesn't work with Windows. However, it is possible to run it with WSL (Windows Subsystem for Linux).

Need help scraping the web?

Then check out **ScrapeOps**, the complete toolkit for web scraping.

Proxy Manager **Scraper Monitoring** **Job Scheduling**

Base Scrapy Project

If you'd like to follow along with a project that is already setup and ready to go you can clone our scrapy project that is made espcially to be used with this tutorial.

Once you download the code from our github repo. You can just copy/paste in the code snippets we use below and see the code working correctly on your computer.

The only thing that you need to do after downloading the code is to install a python virtual environment. If you don't know how to do that you can check out our guide here

If you prefer video tutorials, then check out the video version of this article.

How To Install Scrapy Playwright

Installing scrapy-playwright into your Scrapy projects is very straightforward.

First, you need to install **scrapy-playwright** itself:

```
pip install scrapy-playwright
```

Then if your haven't already installed Playwright itself, you will need to install it using the following command in your command line:

```
playwright install
```

Next, we will need to update our Scrapy projects settings to activate **scrapy-playwright** in the project:

```
# settings.py

DOWNLOAD_HANDLERS = {
    "http": "scrapy_playwright.handler.ScrapyPlaywrightDownloadHandler",
    "https": "scrapy_playwright.handler.ScrapyPlaywrightDownloadHandler",
}

TWISTED_REACTOR = "twisted.internet.asyncioreactor.AsyncioSelectorReactor"
```

The `ScrapyPlaywrightDownloadHandler` class inherits from Scrapy's default `http/https` handler. So unless you explicitly activate **scrapy-playwright** in your Scrapy Request, those requests will be processed by the regular Scrapy download handler.

How To Use Scrapy Playwright In Your Spiders

Now, let's integrate **scrapy-playwright** into a Scrapy spider so all our requests will be JS rendered.

To route our requests through **scrapy-playwright** we just need to enable it in the Request meta dictionary by setting `meta={'playwright': True}`.

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem

class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
                url = "https://quotes.toscrape.com/js/"
                yield scrapy.Request(url, meta={'playwright': True})

        def parse(self, response):
                for quote in response.css('div.quote'):
```

```
                        quote_item = QuoteItem()
                        quote_item['text'] = quote.css('span.text::text').get()
                        quote_item['author'] = quote.css('small.author::text').get()
                        quote_item['tags'] = quote.css('div.tags a.tag::text').getall()
                        yield quote_item
```

The `response` will now contain the rendered page as seen by the browser. However, sometimes Playwright will have ended the rendering before the entire page has been rendered which we can solve using Playwright PageMethods.

Note: If you are getting the following error when running `scrapy crawl`:

```
scrapy.exceptions.NotSupported: Unsupported URL scheme 'https': No module named
'scrapy_playwright'
```

What usually resolves this error is running `deactivate` to deactivate your venv and then re-activate your virtual environment again.

Interacting With The Page Using Playwright PageMethods

To interaction with the page using **scrapy-playwright** we will need to use the `PageMethod` class.

PageMethod's allow us to do alot of different things on the page, including:

- Wait for elements to load before returning response
- Scrolling the page
- Clicking on page elements
- Taking a screenshot of the page
- Creating PDFs of the page

First, to use the **PageMethod** functionality in your spider you will need to set `playwright_include_page` equal to **True** so we can access the Playwright `Page` object and also define any callbacks (i.e. `def parse`) as a coroutine function (`async def`) in order to await the provided `Page` object.

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem


class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
                url = 'https://quotes.toscrape.com/js/'
                yield scrapy.Request(url, meta=dict(
                        playwright = True,
                        playwright_include_page = True,
                ))
```

```
        async def parse(self, response):
                ...
```

Note: When setting `'playwright_include_page': True` it is also recommended that you set a Request errback to make sure pages are closed even if a request fails (if playwright_include_page=False or unset, pages are automatically closed upon encountering an exception).

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem


class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
                url = 'https://quotes.toscrape.com/js/'
                yield scrapy.Request(url, meta=dict(
                        playwright = True,
                        playwright_include_page = True,
                errback=self.errback,
                ))

        async def parse(self, response):
                page = response.meta["playwright_page"]
                await page.close()

                for quote in response.css('div.quote'):
                        quote_item = QuoteItem()
                        quote_item['text'] = quote.css('span.text::text').get()
                        quote_item['author'] = quote.css('small.author::text').get()
                        quote_item['tags'] = quote.css('div.tags a.tag::text').getall()
                        yield quote_item

        async def errback(self, failure):
                page = failure.request.meta["playwright_page"]
                await page.close()
```

---

1. Waiting For Page Elements

To wait for a specific page element before stopping the javascript rendering and returning a response to our scraper we just need to add a `PageMethod` to the `playwright_page_methods` key in out Playwrright settings and define a `wait_for_selector`.

Now, when we run the spider scrapy-playwright will render the page until a **div** with a class **quote** appears on the page.

```
# spiders/quotes.py

import scrapy
```

```
from quotes_js_scraper.items import QuoteItem
from scrapy_playwright.page import PageMethod

class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
                url = "https://quotes.toscrape.com/js/"
                yield scrapy.Request(url, meta=dict(
                                playwright = True,
                                playwright_include_page = True,
                                playwright_page_methods =[PageMethod('wait_for_selector',
'div.quote')],
        errback=self.errback,
                        ))

        async def parse(self, response):
        page = response.meta["playwright_page"]
        await page.close()

                for quote in response.css('div.quote'):
                        quote_item = QuoteItem()
                        quote_item['text'] = quote.css('span.text::text').get()
                        quote_item['author'] = quote.css('small.author::text').get()
                        quote_item['tags'] = quote.css('div.tags a.tag::text').getall()
                        yield quote_item

        async def errback(self, failure):
                page = failure.request.meta["playwright_page"]
                await page.close()
```

---

2. Scraping Multiple Pages

Usually we need to scrape multiple pages on a javascript rendered website. We will do this by checking if there is a next page link present on the page and then requesting that page with the url that we scrape from the page.

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem
from scrapy_playwright.page import PageMethod


class QuotesSpider(scrapy.Spider):
    name = 'quotes'

    def start_requests(self):
        url = "https://quotes.toscrape.com/js/"
        yield scrapy.Request(url, meta=dict(
                playwright = True,
                playwright_include_page = True,
                playwright_page_methods =[
```

```
                    PageMethod('wait_for_selector', 'div.quote'),
                ],
        errback=self.errback,
            ))

    async def parse(self, response):
        page = response.meta["playwright_page"]
        await page.close()

        for quote in response.css('div.quote'):
            quote_item = QuoteItem()
            quote_item['text'] = quote.css('span.text::text').get()
            quote_item['author'] = quote.css('small.author::text').get()
            quote_item['tags'] = quote.css('div.tags a.tag::text').getall()
            yield quote_item

        next_page = response.css('.next>a ::attr(href)').get()

        if next_page is not None:
            next_page_url = 'http://quotes.toscrape.com' + next_page
            yield scrapy.Request(next_page_url, meta=dict(
                playwright = True,
                playwright_include_page = True,
                playwright_page_methods =[
                    PageMethod('wait_for_selector', 'div.quote'),
                ],
                errback=self.errback,
            ))

    async def errback(self, failure):
        page = failure.request.meta["playwright_page"]
        await page.close()
```

---

3. Scroll Down Infinite Scroll Pages

We can also configure scrapy-playwright to scroll down a page when a website uses an infinite scroll to load in data.

In this example, Playwright will wait for `div.quote` to appear, before scrolling down the page until it reachs the 10th quote.

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem
from scrapy_playwright.page import PageMethod

class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
```

```
                url = "https://quotes.toscrape.com/scroll"
                yield scrapy.Request(url, meta=dict(
                                playwright = True,
                                playwright_include_page = True,
                                playwright_page_methods =[
          PageMethod("wait_for_selector", "div.quote"),
          PageMethod("evaluate", "window.scrollBy(0, document.body.scrollHeight)"),
          PageMethod("wait_for_selector", "div.quote:nth-child(11)"),  # 10 per page
          ],
        errback=self.errback,
                        ))

        async def parse(self, response):
        page = response.meta["playwright_page"]
        await page.close()

                for quote in response.css('div.quote'):
                        quote_item = QuoteItem()
                        quote_item['text'] = quote.css('span.text::text').get()
                        quote_item['author'] = quote.css('small.author::text').get()
                        quote_item['tags'] = quote.css('div.tags a.tag::text').getall()
                        yield quote_item

        async def errback(self, failure):
                page = failure.request.meta["playwright_page"]
                await page.close()
```

---

4. Take Screenshot Of Page

Taking screenshots of the page are simple too.

Here we wait for Playwright to see the selector `div.quote` then it takes a screenshot of the page.

```
# spiders/quotes.py

import scrapy
from quotes_js_scraper.items import QuoteItem
from scrapy_playwright.page import PageMethod

class QuotesSpider(scrapy.Spider):
        name = 'quotes'

        def start_requests(self):
                url = "https://quotes.toscrape.com/js/"
                yield scrapy.Request(url, meta=dict(
                                playwright = True,
                                playwright_include_page = True,
                                playwright_page_methods =[
          PageMethod("wait_for_selector", "div.quote"),
          ]
                        ))
```

```
        async def parse(self, response):
        page = response.meta["playwright_page"]
        screenshot = await page.screenshot(path="example.png", full_page=True)
        # screenshot contains the image's bytes
        await page.close()
```

Using Proxies With Scrapy Playwright

In Scrapy Playwright, proxies can be configured at the Browser level by specifying the `proxy` key in the `PLAYWRIGHT_LAUNCH_OPTIONS` setting:

We also need to set the `ignore_https_errors` key in the `playwright_context_kwargs` for the scrapy.Request:

In our example below we will show how it works if you are using the ScrapeOps Proxy API Aggregator. The only part you should have to change is the `YOUR_API_KEY_HERE` - replace this with your scrapeops api key.

(Following code is working as of October 2023)

```
# spiders/quotes.py

import scrapy

class ProxySpider(scrapy.Spider):
    name = "quotes"
    custom_settings = {
        "PLAYWRIGHT_LAUNCH_OPTIONS": {
            "proxy": {
                "server": "http://proxy.scrapeops.io:5353",
                "username": "scrapeops",
                "password": "YOUR_API_KEY_HERE",
            },
        }
    }

    def start_requests(self):
                url = 'https://quotes.toscrape.com/js/'

        yield scrapy.Request(
            "http://httpbin.org/get",
            meta=dict(
                playwright = True,
                playwright_context_kwargs= {
                     "ignore_https_errors": True,
                })
        )

    def parse(self, response):
        print(response.text)
```

More Functionality

Scrapy Playwright has a huge amount of functionality and is highly customisable, so much so that it is hard to cover everything properly in a single guide.

So if you would like to learn more about Scrapy Playwright then check out the offical documentation here.

More Scrapy Tutorials

In this guide we've introduced you to the fundamental functionality of Scrapy Playwright and how to use it in your own projects.

If you would like to learn more about different Javascript rendering options for Scrapy, then be sure to check out our other guides:

- Scrapy JS Rendering Libraries Compared
- Scrapy Splash Guide
- Scrapy Selenium Guide

If you would like to learn more about Scrapy in general, then be sure to check out The Scrapy Playbook.